

Leonardo Tomasela Leal
170291

Projeto de um Processador RISC-V

São José dos Campos - Brasil
Abril de 2026

Leonardo Tomasela Leal
170291

Projeto de um Processador RISC-V

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina de Laboratório de Sistemas Computacionais: Arquitetura e Organização de Computadores.

Docente: Prof. Dr. Sérgio Ronaldo Barros dos Santos

Universidade Federal de São Paulo - UNIFESP

Instituto de Ciência e Tecnologia - Campus São José dos Campos

São José dos Campos - Brasil

Abril de 2026

Resumo

Na disciplina de Arquitetura e Organização de Computadores foram abordados vários conceitos sobre o projeto e o funcionamento de processadores, neste documento estes conhecimentos são colocados em prática na forma da elaboração do projeto de um processador *RISC-V* de 32 bits, com 32 registradores de propósito geral.

Palavras-chaves: Arquitetura e Organização de Computadores. RISC-V. Kit FPGA. ISA. Datapath. Caminho de dados. Conjunto de instruções.

Lista de ilustrações

Figura 1 – Arquitetura de Von Neumann e de Harvard	11
Figura 2 – Endereçamento por registrador	12
Figura 3 – Endereçamento por imediato	13
Figura 4 – Endereçamento por deslocamento	13
Figura 5 – Endereçamento PC-relativo	14
Figura 6 – Caminho de Dados Completo	21
Figura 7 – Caminho de Dados das Instruções do Tipo R	22
Figura 8 – Caminho de Dados das Instruções Aritméticas e Lógicas com Imediato	23
Figura 9 – Caminho de Dados das Instruções de Carregamento	24
Figura 10 – Caminho de Dados das Instruções de Armazenamento	25
Figura 11 – Caminho de Dados das Instruções de Desvio Condicional	26
Figura 12 – Caminho de Dados da Instrução de Salto e Ligação	27
Figura 13 – Caminho de Dados da Instrução de Salto Indireto	28
Figura 14 – Caminho de Dados de Carregamento de Imediato Superior	29
Figura 15 – Caminho de Dados da Adição de Imediato Superior ao PC	30

Lista de tabelas

Tabela 1 – Tipos de instruções	17
Tabela 2 – Instruções do tipo R	17
Tabela 3 – Instruções do tipo I	18
Tabela 4 – Instruções do tipo S	19
Tabela 5 – Instruções do tipo B	19
Tabela 6 – Instruções do tipo U	20

Sumário

1	INTRODUÇÃO	7
2	OBJETIVOS	8
2.1	Geral	8
2.2	Específico	8
3	FUNDAMENTAÇÃO TEÓRICA	9
3.1	Computador	9
3.2	Organização de Computadores	9
3.3	Arquitetura de Computadores	9
3.3.1	CISC vs RISC	9
3.3.1.1	Arquitetura CISC (Complex Instruction Set Computer)	10
3.3.1.2	Arquitetura RISC (Reduced Instruction Set Computer)	10
3.3.2	Arquitetura de Von Neumann vs. Arquitetura de Harvard	10
3.3.2.1	Arquitetura de Von Neumann	11
3.3.2.2	Arquitetura de Harvard	11
3.3.3	Modos de Endereçamento	12
3.3.3.1	Endereçamento por registrador	12
3.3.3.2	Endereçamento por imediato	13
3.3.3.3	Endereçamento por deslocamento	13
3.3.3.4	Endereçamento PC-relativo	13
3.3.4	Arquitetura RISC-V	14
3.3.4.1	Origem	14
3.3.4.2	Características	14
3.3.4.3	Modos de Endereçamento	15
3.3.5	Justificativa da Escolha	15
4	DESENVOLVIMENTO	16
4.1	Características Gerais do Processador	16
4.2	Desenvolvimento da Arquitetura	16
4.2.1	Instruções do Tipo R	17
4.2.2	Instruções do Tipo I	18
4.2.3	Instruções do Tipo S	18
4.2.4	Instruções do Tipo B	19
4.2.5	Instruções do Tipo U	19
4.3	Desenvolvimento da Organização	20

4.3.1	Caminho de Dados Completo	21
4.3.2	Caminho de Dados das Instruções do Tipo R	21
4.3.3	Caminho de Dados das Instruções Aritméticas e Lógicas com Imediato (<i>addi, xori, ori, andi, slli, srli, srai, slti, sltiu</i>)	22
4.3.4	Caminho de Dados das Instruções de Carregamento (<i>lb, lh, lw, lbu, lhu</i>)	23
4.3.5	Caminho de Dados das Instruções de Armazenamento (<i>sb, sh, sw</i>)	24
4.3.6	Caminho de Dados das Instruções de Desvio Condicional (<i>beq, bne, blt, bge, bltu, bgeu</i>)	25
4.3.7	Caminho de Dados da Instrução de Salto e Ligação (<i>jal</i>)	26
4.3.8	Caminho de Dados da Instrução de Salto Indireto (<i>jalr</i>)	27
4.3.9	Caminho de Dados de Carregamento de Imediato Superior (<i>lui</i>)	28
4.3.10	Caminho de Dados da Adição de Imediato Superior ao PC (<i>auipc</i>)	29
5	CONSIDERAÇÕES FINAIS	31
	REFERÊNCIAS	32

1 Introdução

"Os computadores levaram a uma terceira revolução para a civilização, com a revolução da informação tomando o seu lugar ao lado da agricultura e revoluções industriais." (PATTERSON; HENNESSY, 2018, p. 41, tradução própria). A evolução dos sistemas computacionais está intrinsecamente ligada ao constante aprimoramento das suas arquiteturas e da sua organização física. O estudo prático do funcionamento lógico de um processador apresenta-se como uma etapa indispensável para a consolidação teórica dos conceitos abordados. Neste contexto, o presente relatório documenta o projeto de um processador digital baseado na arquitetura RISC-V de 32 bits, operando sob um modelo de execução de ciclo único (monociclo).

A motivação central para a adoção da norma RISC-V no desenvolvimento deste projeto reside na sua natureza de padrão de código aberto (open source) e na sua licença totalmente gratuita. Estas características posicionam o RISC-V como uma alternativa acadêmica e industrial altamente viável e moderna face a arquiteturas proprietárias e fechadas que dominam o mercado atual, tais como as famílias x86 e ARM. Para além da acessibilidade financeira e filosófica, o RISC-V destaca-se pela sua profunda modularidade. O seu conjunto de instruções foi estrategicamente desenhado para ser enxuto na base, sendo suportado por várias extensões normalizadas, o que o torna fácil de adaptar para diversos usos de processamento. Esta modularidade é o que viabiliza a inclusão, neste processador, de um bloco específico de multiplicação e divisão em hardware acoplado à base de inteiros (constituindo um subconjunto da variante RV32IM).

Por conseguinte, o objetivo central deste documento é expor, de forma rigorosa, as decisões arquiteturais e organizacionais tomadas durante a elaboração do hardware. O relatório encontra-se estruturado de modo a apresentar, numa primeira instância, os objetivos específicos do projeto, seguidos de uma fundamentação teórica sólida sobre os paradigmas computacionais. Posteriormente, são detalhados o subconjunto de instruções implementado e os seus respectivos formatos, culminando na apresentação do desenvolvimento da organização estrutural, onde se discriminam os componentes do sistema e o desenho do caminho de dados (datapath).

...

2 Objetivos

2.1 Geral

Estabelecer as especificações teóricas e a organização de um processador baseado na arquitetura RISC-V, aplicando de forma prática os conceitos de arquitetura e organização de sistemas computacionais adquiridos na disciplina de Arquitetura e Organização de Computadores. Tal processador será posteriormente implementado via linguagem de descrição de Hardware.

2.2 Específico

Para a composição deste relatório inicial, os seguintes marcos foram definidos como objetivos centrais:

1. Definir o conjunto de instruções e seus modos de endereçamento, seguindo os requisitos da arquitetura RISC-V.
2. Estabelecer a organização estrutural dos blocos funcionais do processador.
3. Especificar o caminho de dados de cada instrução.
4. Adaptar a arquitetura RISC-V conforme as necessidades do projeto.

3 Fundamentação Teórica

Esta seção tem como objetivo estabelecer a base teórica e conceitual que fundamenta o desenvolvimento do projeto. Nela, são abordados os princípios fundamentais de arquitetura e organização de computadores, os paradigmas de memória e modos de endereçamento, bem como as principais características da arquitetura RISC-V, fornecendo o embasamento literário necessário para justificar as escolhas de projeto adotadas.

3.1 Computador

Define-se, em sua essência, como um sistema digital e eletrônico projetado para receber, armazenar, processar e transmitir informações de maneira automatizada, operando sob o controle estrito de um conjunto de instruções previamente armazenado.

3.2 Organização de Computadores

Refere-se à implementação física e de hardware responsável por concretizar as especificações da arquitetura proposta. O projeto organizacional envolve o arranjo das unidades operacionais e suas interconexões lógicas, englobando o desenho do caminho de dados (datapath), o dimensionamento dos sinais da Unidade de Controle e a hierarquia topológica do sistema de memórias.

3.3 Arquitetura de Computadores

Corresponde aos atributos lógicos de um sistema computacional que são diretamente visíveis ao programador de baixo nível ou ao desenvolvedor de compiladores. Ela engloba a definição formal do Conjunto de Instruções (ISA - Instruction Set Architecture), os tipos de dados matemáticos suportados, a quantidade de registradores de uso geral expostos e os métodos padronizados de acesso à memória. Em suma, a arquitetura estabelece as especificações funcionais que determinam o que o processador é capaz de realizar.

3.3.1 CISC vs RISC

A distinção entre os paradigmas CISC (Complex Instruction Set Computer) e RISC (Reduced Instruction Set Computer) é fundamental na classificação de processadores.

3.3.1.1 Arquitetura CISC (Complex Instruction Set Computer)

A arquitetura CISC caracteriza-se por um conjunto amplo e heterogêneo de instruções, com tamanhos variáveis e múltiplos modos de endereçamento. Neste modelo, uma única instrução pode realizar várias operações de baixo nível (como acessar a memória, processar o dado e guardar o resultado). Embora reduza o tamanho do código compilado, esta abordagem aumenta substancialmente a complexidade do hardware e da Unidade de Controle, resultando em ciclos de execução irregulares.

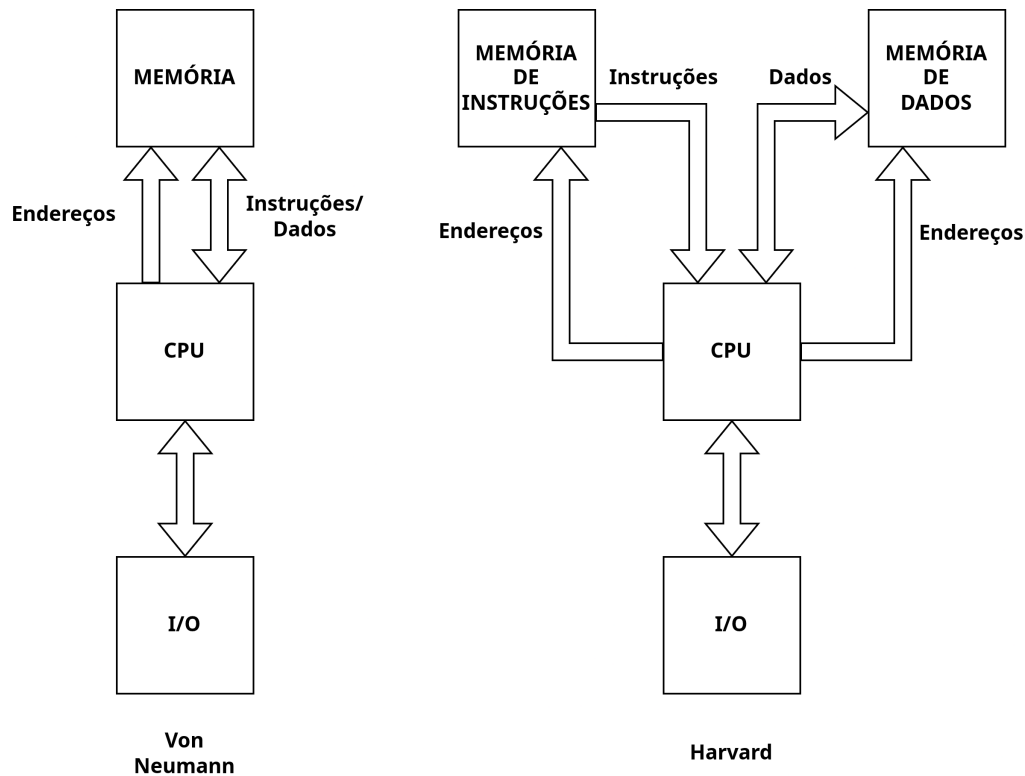
3.3.1.2 Arquitetura RISC (Reduced Instruction Set Computer)

A arquitetura RISC adota um conjunto reduzido de instruções simples, otimizadas e de formato fixo, o que agiliza drasticamente a decodificação. A sua característica estrutural central é a adoção rigorosa do modelo load-store, em que o processamento matemático e lógico ocorre exclusivamente entre os registradores internos. O acesso à memória principal é estritamente restrito a instruções dedicadas de leitura (load) e escrita (store). Esta filosofia simplifica o hardware, reduz os modos de endereçamento e garante a estabilidade previsível essencial para viabilizar a implementação de um caminho de dados de ciclo único.

3.3.2 Arquitetura de Von Neumann vs. Arquitetura de Harvard

A Figura 1 apresenta as diferenças entre as arquiteturas, que serão elaboradas nos subtópicos seguintes.

Figura 1 – Arquitetura de Von Neumann e de Harvard



Fonte: O Autor

3.3.2.1 Arquitetura de Von Neumann

Fundamentada no conceito clássico de programa armazenado, este modelo caracteriza-se por manter a memória de dados e a memória de instruções estruturadas de forma unificada. Consequentemente, o sistema dispõe de um barramento único para o tráfego de informações. Essa característica estrutural cria um afunilamento (conhecido como "Gargalo de Von Neumann"), que obriga o processador a executar as instruções em uma ordem estritamente sequencial, impossibilitando a busca de uma instrução e a manipulação de um dado no mesmo instante de tempo. É um padrão habitualmente associado aos processadores de instrução complexa (CISC).

3.3.2.2 Arquitetura de Harvard

Em contrapartida, este paradigma resolve o gargalo de concorrência ao adotar a separação física estrita entre a memória de dados e a memória de instruções. A utilização de barramentos de comunicação isolados possibilita que o processador efetue a busca da próxima instrução e a leitura ou escrita de um dado operacional de forma simultânea. Esta abordagem de acesso paralelo é inerente ao modelo RISC e torna-se um requisito vital para a implementação de um caminho de dados de ciclo único.

3.3.3 Modos de Endereçamento

Os modos de endereçamento consistem nos mecanismos lógicos e aritméticos pelos quais a arquitetura do hardware calcula os endereços efetivos dos operandos requisitados para processar uma instrução. Para esta seção utilizaremos a seguinte notação, baseada na utilizada por Stallings ([STALLINGS, 2017](#)) para explicar os modos de endereçamento:

1. **A** $\Rightarrow$ O conteúdo no campo de endereço dentro de uma instrução.
2. **PC** $\Rightarrow$ O conteúdo em uma instrução se refere ao registrador contador de programa.
3. **R** $\Rightarrow$ O conteúdo em uma instrução se refere a um registrador.
4. **(X)** $\Rightarrow$ O conteúdo do local de memória X ou do registrador X.
5. **EA** $\Rightarrow$ Endereço real do operando.

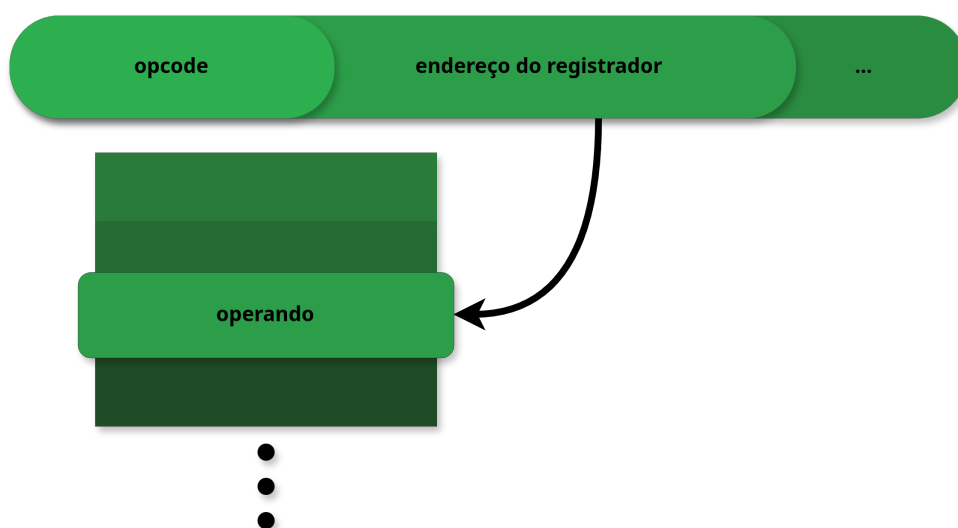
O escopo do projeto sustenta-se nos seguintes modos principais:

3.3.3.1 Endereçamento por registrador

A própria instrução decodificada faz uma referência direta ao endereço numérico de um registrador interno, que, por sua vez, armazena o operando necessário para a Unidade Lógica e Aritmética:

$$EA = R$$

Figura 2 – Endereçamento por registrador



Fonte: O Autor

3.3.3.2 Endereçamento por imediato

O valor numérico constante a ser utilizado na operação não é buscado no banco de registradores, pois já se encontra embutido fisicamente em um campo específico dos próprios bits da instrução:

$$EA = A$$

Figura 3 – Endereçamento por imediato



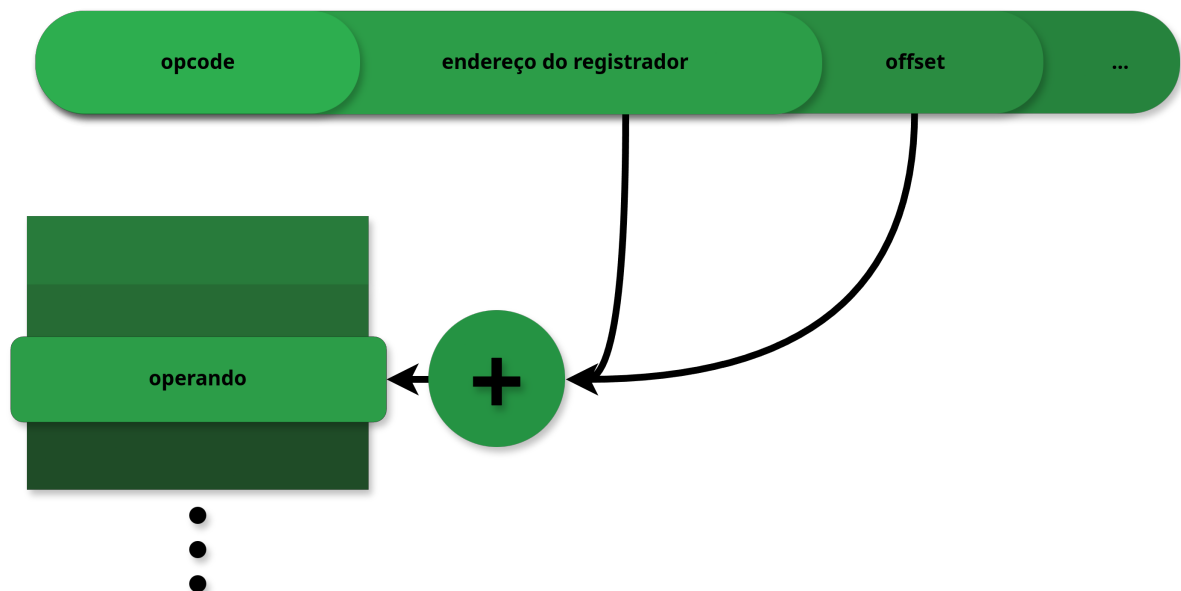
Fonte: O Autor

3.3.3.3 Endereçamento por deslocamento

O processo de acesso ao módulo de memória de dados é estabelecido somando-se um valor referencial de base (armazenado em um registrador) a um valor fixo de deslocamento (caracterizado como um offset imediato de 12 bits na ISA abordada):

$$EA = A + (R)$$

Figura 4 – Endereçamento por deslocamento



Fonte: O Autor

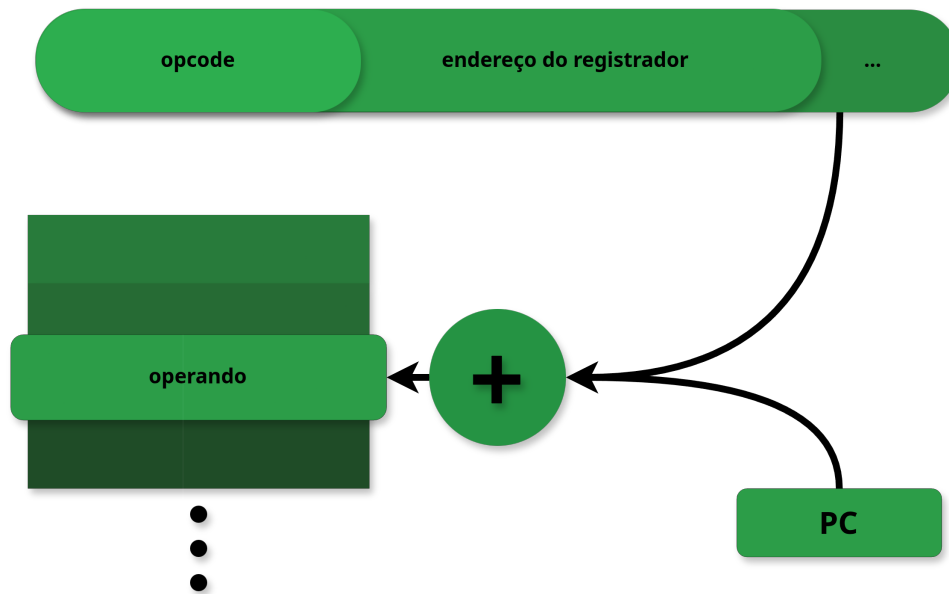
3.3.3.4 Endereçamento PC-relativo

Empregado no fluxo de controle de saltos e desvios, o endereço computado para o salto é obtido somando-se o valor atual armazenado no Contador de Programa (PC) a um

valor imediato extraído da instrução:

$$EA = A + (PC)$$

Figura 5 – Endereçamento PC-relativo



Fonte: O Autor

3.3.4 Arquitetura RISC-V

3.3.4.1 Origem

O RISC-V foi concebido e estruturado na Universidade da Califórnia em Berkeley (UC Berkeley) ao longo do verão do ano de 2010. O projeto foi idealizado e gerido por uma equipe de acadêmicos e pesquisadores englobando pioneiros da computação como Krste Asanović e David Patterson. O desenvolvimento se originou da necessidade intrínseca da equipe por um processador de arquitetura moderna e de licença livre (gratuita), passível de ser implementado sem restrições em aulas práticas e pesquisas.

3.3.4.2 Características

O RISC-V é uma arquitetura estabelecida sobre os preceitos do modelo RISC (Reduced Instruction Set Computer). Diferenciando-se das arquiteturas complexas (CISC), ele é construído sobre o arranjo load-store (onde o acesso físico à memória restringe-se a instruções exclusivas de leitura e escrita), disponibiliza um largo conjunto de 32 registradores de uso geral e processa instruções com formato simples e de tamanho rigidamente fixado em 32 bits. O sistema utiliza um mapeamento de endereçamento com formato padrão orientado a byte de acordo com a ordem little endian ([RISC-V INTERNATIONAL, 2024](#)).

3.3.4.3 Modos de Endereçamento

Mantendo plena conformidade com sua filosofia de redução e simplificação computacional (cujo foco preza pela minimização da complexidade dos decodificadores), o conjunto de instruções do RISC-V opera unicamente sob o emprego dos modos de endereçamento por registrador (para manipulações puramente lógicas/matemáticas), por imediato (para interações com constantes numéricas), por deslocamento (recurso fundamental das instruções nativas de Load e Store) e de maneira PC-relativa (encarregado pelo direcionamento imperativo de saltos e desvios estruturados) ([RISC-V INTERNATIONAL, 2024](#)).

3.3.5 Justificativa da Escolha

A escolha definitiva da ISA RISC-V para conduzir o desenvolvimento lógico exposto neste relatório consolida-se, fundamentalmente, pelas garantias de seu padrão de código aberto (open source) atrelado à abstenção total de pagamentos e taxas de licenciamento comercial (gratuito). Essas propriedades proporcionam uma viabilidade sem precedentes e firmam a arquitetura como uma alternativa moderna, acadêmica e industrialmente proeminente frente aos competidores corporativos preestabelecidos, tais como os processadores das famílias x86 e ARM. Além dos benefícios econômicos e da disponibilidade, a base técnica justifica-se pela adoção de um design profundamente modular. O RISC-V disponibiliza um conjunto essencial de instruções rigorosamente simplificadas, sendo projetado de tal forma que é perfeitamente exequível adaptá-lo a variados cenários mediante o simples acoplamento de extensões. Essa prerrogativa arquitetural viabiliza e acomoda a inclusão direcionada do bloco de multiplicação em hardware abordada neste projeto.

4 Desenvolvimento

Neste capítulo, detalha-se o processo de concepção do processador, abrangendo desde as definições macroestruturais até a especificação técnica das instruções e a lógica de organização do hardware.

4.1 Características Gerais do Processador

O processador desenvolvido é um núcleo de processamento digital baseado na arquitetura RISC-V, implementando um subconjunto do conjunto de instruções RV32IM. As principais diretrizes técnicas que definem o sistema são:

1. **Largura de Dados:** 32 bits, operando com palavras (words) de quatro bytes.
2. **Modelo de Execução:** Monociclo (Ciclo Único), onde cada instrução é buscada, decodificada e executada em um único pulso de clock.
3. **Conjunto de Instruções (ISA):** Base inteira (I) com a extensão de multiplicação e divisão em hardware (M). Não está planejada a implementação das instruções *ebreak* e *ecall*, entretanto, caso necessário, isso pode mudar.
4. **Ordenação de Memória:** Padrão Little-endian, com endereçamento orientado a byte.
5. **Banco de Registradores:** Composto por 32 registradores de uso geral (x0 a x31), cada um com 32 bits de largura.
6. **Arquitetura de Memória:** Modelo Harvard, com a separação física entre a memória de instruções e a memória de dados para viabilizar o fluxo de dados em ciclo único.

4.2 Desenvolvimento da Arquitetura

A arquitetura RISC-V organiza suas instruções em formatos fixos de 32 bits para otimizar a decodificação em hardware ([RISC-V INTERNATIONAL, 2024](#)). Para este projeto, as instruções foram agrupadas em cinco categorias principais:

Tabela 1 – Tipos de instruções

31	25	24	20	19	15	14	12	11	7	6	0	
funct7		rs2		rs1		funct3		rd		opcode		Tipo R
imm[11:0]				rs1		funct3		rd		opcode		Tipo I
imm[11:5]		rs2		rs1		funct3		imm[4:0]		opcode		Tipo S
imm[11:5]		rs2		rs1		funct3		imm[4:0]		opcode		Tipo B
imm[31:12]										opcode		Tipo U

Fonte: O Autor

4.2.1 Instruções do Tipo R

As instruções do Tipo R são projetadas para a execução de operações aritméticas e lógicas nas quais todos os operandos envolvidos residem internamente no banco de registradores. Consequentemente, esta categoria utiliza de forma exclusiva o modo de endereçamento por registrador, lendo os dados de duas fontes e gravando o resultado em um destino. Como exemplos práticos desta categoria implementados no projeto, destacam-se as operações lógicas e aritméticas fundamentais (como soma, subtração, operações lógicas and, or e xor), os deslocamentos de bits, as comparações de magnitude e, de modo integrado, as operações atreladas à Extensão M de hardware, compreendendo as funções de multiplicação e divisão em nível de silício.

Tabela 2 – Instruções do tipo R

Mnemônico	funct3	funct7	Opcode	Descrição	Notas
add	0x0	0x00	0110011	$rd = rs1 + rs2$	
sub	0x0	0x20	0110011	$rd = rs1 - rs2$	
xor	0x4	0x00	0110011	$rd = rs1 \wedge rs2$	
or	0x6	0x00	0110011	$rd = rs1 \mid rs2$	
and	0x7	0x00	0110011	$rd = rs1 \& rs2$	
sll	0x1	0x00	0110011	$rd = rs1 \ll rs2$	
srl	0x5	0x00	0110011	$rd = rs1 \gg rs2$	
sra	0x5	0x20	0110011	$rd = rs1 \gg rs2$	msb-extends
slt	0x2	0x00	0110011	$rd = (rs1 < rs2)?1:0$	
sltu	0x3	0x00	0110011	$rd = (rs1 < rs2)?1:0$	zero-extends
mul	0x0	0x01	0110011	$rd = (rs1 * rs2)[31:0]$	
mulh	0x1	0x01	0110011	$rd = (rs1 * rs2)[63:32]$	
mulsu	0x2	0x01	0110011	$rd = (rs1 * rs2)[63:32]$	
mulu	0x3	0x01	0110011	$rd = (rs1 * rs2)[63:32]$	
div	0x4	0x01	0110011	$rd = rs1 / rs2$	
divu	0x5	0x01	0110011	$rd = rs1 / rs2$	
rem	0x6	0x01	0110011	$rd = rs1 \% rs2$	
remu	0x7	0x01	0110011	$rd = rs1 \% rs2$	

Fonte: O Autor

É importante destacar o comportamento de funções que trabalham com valores *unsigned*, valores que não apresentam sinais e que possuem características específicas, relacionadas a como eles são tratados quando estendidos. Tais comportamentos são

descritos na coluna "Notas". A descrição *msb-extends* diz que o bit mais significativo é repetido quanto o valor original é estendido, já a descrição *zero-extends* diz que o valor é completado por zeros quando é estendido. Essa afirmação também vale para tipos posteriores de instrução.

4.2.2 Instruções do Tipo I

As instruções do Tipo I (Immediate) caracterizam-se por operar combinando um registrador de origem a um valor constante (imediato) de 12 bits embutido na própria instrução, o qual sofre extensão de sinal prévia à sua utilização. Este formato é versátil e emprega dois modos de endereçamento distintos a depender da operação: utiliza o endereçamento por imediato para a execução de instruções lógico-aritméticas com constantes (a exemplo da soma com imediato) e o endereçamento por deslocamento para as instruções voltadas à leitura da memória de dados, utilizando o imediato como um referencial de distância (como nas operações de carregamento de palavra, meia palavra e byte). Além disso, este formato atende às necessidades de saltos indiretos realizados por valores de registradores.

Tabela 3 – Instruções do tipo I

Mnemônico	funct3	imm	Opcode	Descrição	Notas
addi	0x0		0010011	$rd = rs1 + imm$	
xori	0x4		0010011	$rd = rs1 \wedge imm$	
ori	0x6		0010011	$rd = rs1 \mid imm$	
andi	0x7		0010011	$rd = rs1 \& imm$	
slli	0x1	$imm[5:11]=0x00$	0010011	$rd = rs1 \ll imm[0:4]$	
srlr	0x5	$imm[5:11]=0x00$	0010011	$rd = rs1 \gg imm[0:4]$	
srai	0x5	$imm[5:11]=0x20$	0010011	$rd = rs1 \gg imm[0:4]$	msb-extends
slti	0x2		0010011	$rd = (rs1 < imm)?1:0$	
sltiu	0x3		0010011	$rd = (rs1 < imm)?1:0$	zero-extends
lb	0x0		0000011	$rd = M[rs1+imm][0:7]$	
lh	0x1		0000011	$rd = M[rs1+imm][0:15]$	
lw	0x2		0000011	$rd = M[rs1+imm][0:31]$	
lbu	0x4		0000011	$rd = M[rs1+imm][0:7]$	zero-extends
lhu	0x5		0000011	$rd = M[rs1+imm][0:15]$	zero-extends
jalr	0x0		1100111	$rd = PC+4; PC = rs1 + imm$	

Fonte: O Autor

4.2.3 Instruções do Tipo S

Para as operações de escrita, empregam-se as instruções do Tipo S (Store), que representam o formato exclusivo para a transferência de dados do processador para a memória externa. Estas operações fazem uso estrito do modo de endereçamento por deslocamento, no qual o endereço de destino na memória é computado através da soma do valor contido em um registrador base com um deslocamento imediato de 12 bits. Diferentemente das categorias anteriores, o Tipo S não requer um registrador de destino

para gravação interna, uma vez que o dado operado é enviado ao sistema de armazenamento. Exemplos desta implementação englobam as operações de salvamento de palavra inteira, frações de palavra e unidades de byte.

Tabela 4 – Instruções do tipo S

Mnemônico	funct3	Opcode	Descrição
sb	0x0	0100011	$M[rs1+imm][0:7] = rs2[0:7]$
sh	0x1	0100011	$M[rs1+imm][0:15] = rs2[0:15]$
sw	0x2	0100011	$M[rs1+imm][0:31] = rs2[0:31]$

Fonte: O Autor

4.2.4 Instruções do Tipo B

No que tange à manipulação do fluxo de controle condicional, o sistema adota as instruções do Tipo B (Branch). Estas instruções operam calcadas no modo de endereçamento PC-relativo e são responsáveis por realizar avaliações lógicas entre dois registradores. Caso a condição avaliada seja considerada verdadeira, o processador efetua o desvio do fluxo de execução para um novo endereço, o qual é calculado mediante a soma do valor atual do Contador de Programa (PC) a um valor imediato especificado na instrução. A implementação funcional deste projeto contempla saltos condicionais baseados em relações de igualdade, desigualdade e comparações de magnitude relativas.

Tabela 5 – Instruções do tipo B

Mnemônico	funct3	Opcode	Descrição	Notas
beq	0x0	1100011	$\text{if}(rs1 == rs2) \text{ PC } += \text{imm}$	
bne	0x1	1100011	$\text{if}(rs1 != rs2) \text{ PC } += \text{imm}$	
blt	0x4	1100011	$\text{if}(rs1 < rs2) \text{ PC } += \text{imm}$	
bge	0x5	1100011	$\text{if}(rs1 \geq rs2) \text{ PC } += \text{imm}$	
bltu	0x6	1100011	$\text{if}(rs1 < rs2) \text{ PC } += \text{imm}$	zero-extends
bgeu	0x7	1100011	$\text{if}(rs1 \geq rs2) \text{ PC } += \text{imm}$	zero-extends

Fonte: O Autor

4.2.5 Instruções do Tipo U

Por fim, as instruções do Tipo U (Upper Immediate) são estruturadas para manipular os 20 bits mais significativos de uma palavra, viabilizando a construção de constantes numéricas extensas e o desvio de longo alcance. Empregando os modos de endereçamento por imediato e PC-relativo, este formato permite o carregamento de valores superiores diretos e a adição de imediatos superiores ao Contador de Programa. Conforme o adendo de projeto supracitado, esta categoria também absorveu as responsabilidades dos saltos incondicionais tradicionais, integrando a lógica de salto e ligação à sua infraestrutura de imediatos estendidos.

Tabela 6 – Instruções do tipo U

Mnemônico	Opcode	Descrição
lui	0110111	rd = imm << 12
auipc	0010111	rd = PC + (imm << 12)
jal	1101111	rd = PC+4; PC += imm

Fonte: O Autor

Vale a pena ressaltar que na versão original do RISC-V, a instrução de *jal* era caracterizada como "tipo J", mas por simplicidade, neste projeto ela é caracterizada como tipo U, na versão 2.0 da arquitetura RISC-V a instrução *jal* também foi modificada para tipo U, mas em versões posteriores ela volta a ser caracterizada como tipo J. (ZICO, 2018)

4.3 Desenvolvimento da Organização

A organização do processador reflete a materialização física da arquitetura previamente definida, feita por meio da construção dos caminhos de dados de cada instrução, como da descrição do mesmo. Em cada componente funcional, há 4 campos:

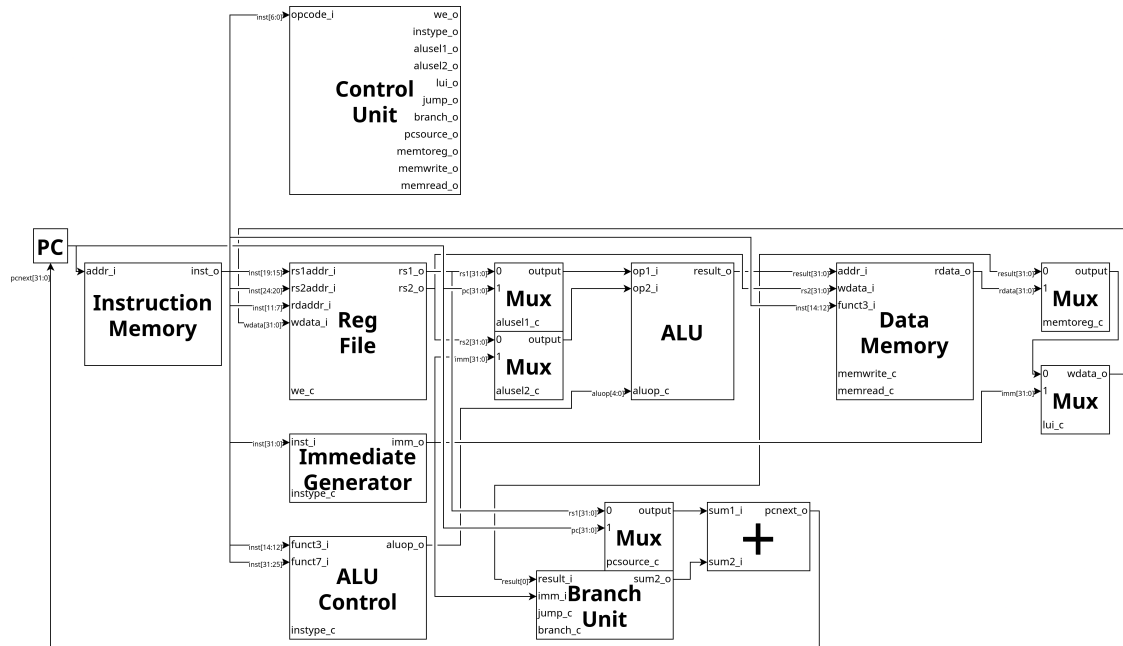
1. **O nome do componente funcional**, localizado no centro de cada caixa.
2. **As entradas gerais de cada componente funcional**, localizados no canto superior esquerdo de cada caixa e com nomes terminados em "*i*".
3. **As entradas de controle de cada componente funcional**, localizadas no canto inferior esquerdo de cada caixa e com nomes terminados em "*c*".
4. **As saídas de cada componente funcional**, localizadas na parte direita de cada caixa, com nomes terminados em "*o*".

Vale ressaltar que para uma melhor compreensão de cada caminho de dados, foram omitidos os caminhos de controle com origem na unidade de controle (*Control Unit*), sendo representados por um texto em vermelho quando ativos, tanto na saída da unidade de controle quanto na entrada de controle dos componentes funcionais. Os caminhos são interligados por entradas e saídas com o mesmo nome antes do caractere "*_*".

Nos caminhos de dados de instruções específicas, além dos caminhos de controle, foram omitidos os caminhos não utilizados.

4.3.1 Caminho de Dados Completo

Figura 6 – Caminho de Dados Completo

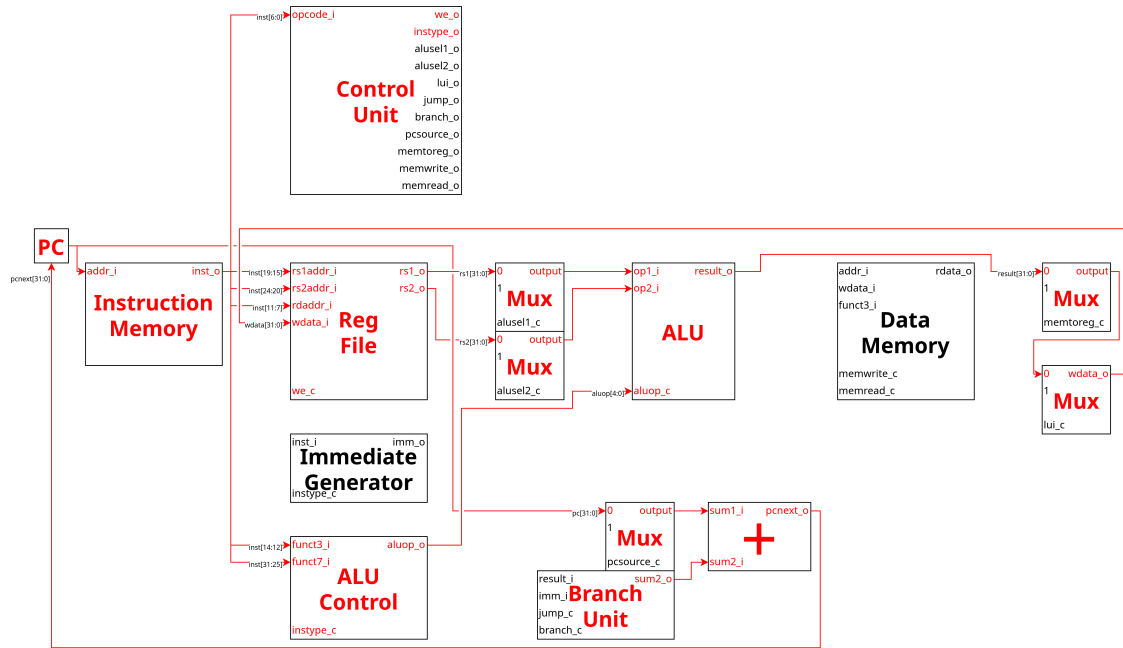


Fonte: O Autor

4.3.2 Caminho de Dados das Instruções do Tipo R

Para as instruções estritamente dependentes de registradores, o fluxo de dados inicia-se com a leitura simultânea dos operandos armazenados nos endereços apontados pelos campos *rs1* e *rs2* do Banco de Registradores. A Unidade de Controle sinaliza ao multiplexador da ALU que o segundo operando deve provir diretamente de *rs2*, ignorando qualquer valor imediato. A ALU executa a operação matemática ou lógica estipulada pela ALU Control, que define a operação com base nos campos originados da instrução e, em seguida, o resultado transita pelos multiplexadores de saída (ignorando a Memória de Dados) para ser gravado de volta no registrador de destino (*rd*).

Figura 7 – Caminho de Dados das Instruções do Tipo R

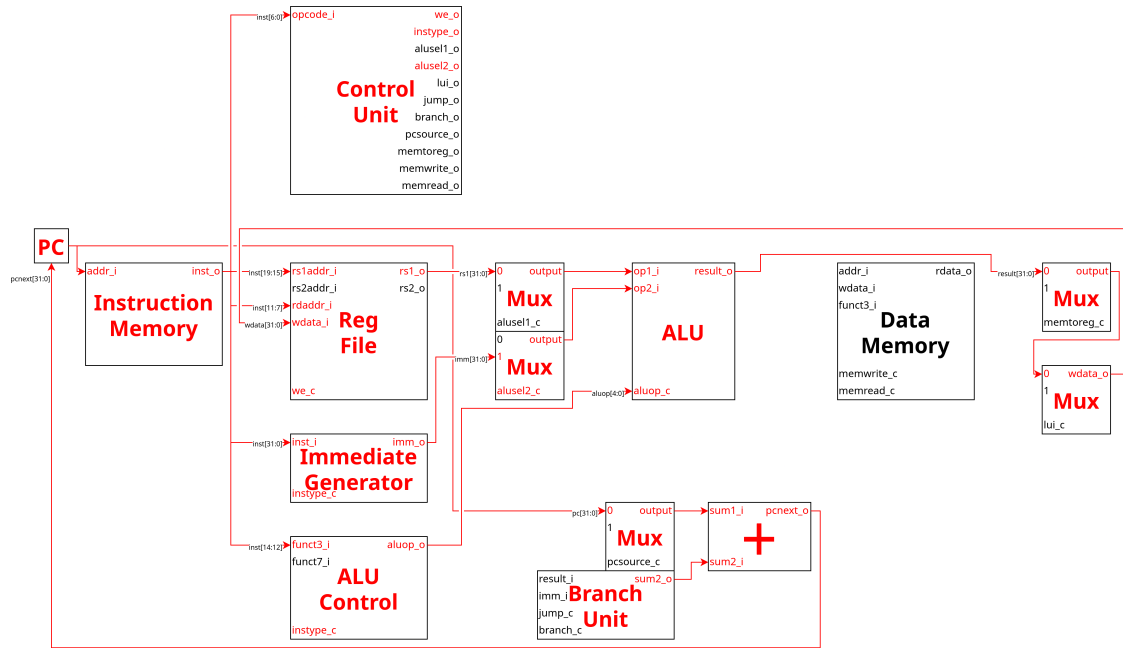


Fonte: O Autor

4.3.3 Caminho de Dados das Instruções Aritméticas e Lógicas com Imediato (*addi, xori, ori, andi, slli, srli, srai, slti, sltiu*)

Neste agrupamento, o datapath opera lendo apenas um registrador de origem (*rs1*). Paralelamente, o Gerador de Imediatos extrai o valor constante de 12 bits embutido na instrução e realiza a extensão de sinal para 32 bits. O multiplexador de entrada da ALU é então comutado para selecionar este valor estendido como o segundo operando. Após o processamento da operação pela ALU, o resultado é diretamente encaminhado para a porta de escrita do Banco de Registradores, sendo salvo no registrador *rd*.

Figura 8 – Caminho de Dados das Instruções Aritméticas e Lógicas com Imediato



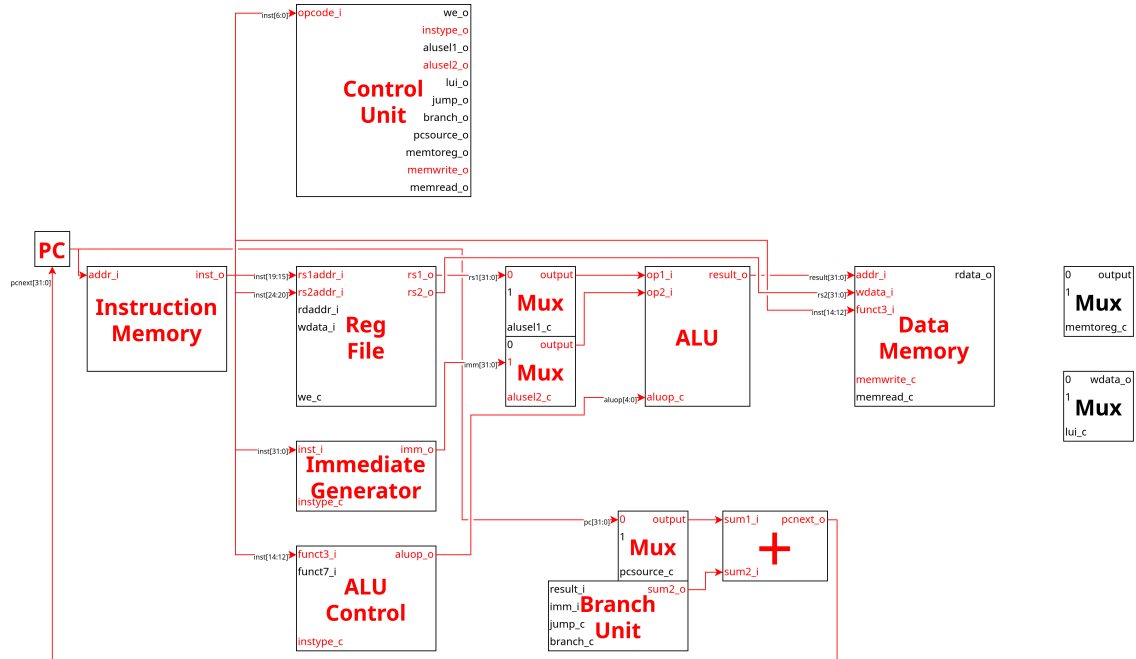
Fonte: O Autor

4.3.4 Caminho de Dados das Instruções de Carregamento (*lb*, *lh*, *lw*, *lbu*, *lhu*)

O fluxo de dados para a leitura em memória exige a utilização da ALU para o cálculo do endereço efetivo. O valor base é lido do registrador *rs1* e somado ao valor de deslocamento (imediato de 12 bits estendido). O resultado desta soma é enviado para a porta de endereço da Memória de Dados. Com o sinal de leitura ativado, a memória disponibiliza o dado requisitado. O multiplexador de write-back é então configurado para selecionar a saída da Memória de Dados, redirecionando o valor lido para ser armazenado no registrador de destino (*rd*).

É importante ressaltar que para as instruções sem bit de sinal *lbu* e *lhu*, a extensão do valor é realizado dentro da própria memória de dados. O reconhecimento de tais instruções é realizado de mesma forma do reconhecimento das demais instruções de carregamento, utilizando-se do campo *funct3* da instrução.

Figura 10 – Caminho de Dados das Instruções de Armazenamento

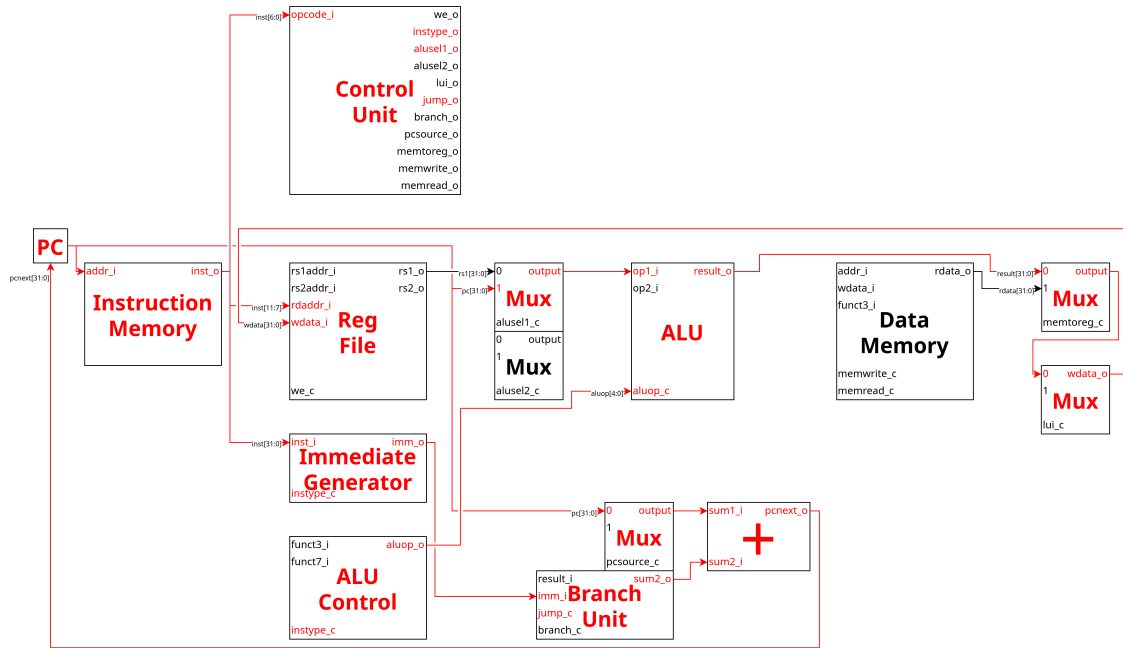


Fonte: O Autor

4.3.6 Caminho de Dados das Instruções de Desvio Condicional (*beq*, *bne*, *blt*, *bge*, *bltu*, *bgeu*)

Nas instruções de salto condicional, o caminho de dados bifurca-se em duas operações simultâneas. Na primeira via, os valores dos registradores *rs1* e *rs2* são enviados à ALU (ou a um comparador dedicado) para avaliar a condição lógica estipulada. Na segunda via, um somador independente calcula o endereço de destino do salto, adicionando o valor atual do Contador de Programa (PC) ao imediato de 12 bits estendido (e deslocado à esquerda). Se a condição avaliada pela ALU for verdadeira, a Unidade de Controle comuta o multiplexador do PC para que o próximo endereço de instrução seja o alvo calculado; caso contrário, o PC é atualizado normalmente para $PC + 4$. Não ocorre gravação no Banco de Registradores.

Figura 12 – Caminho de Dados da Instrução de Salto e Ligação

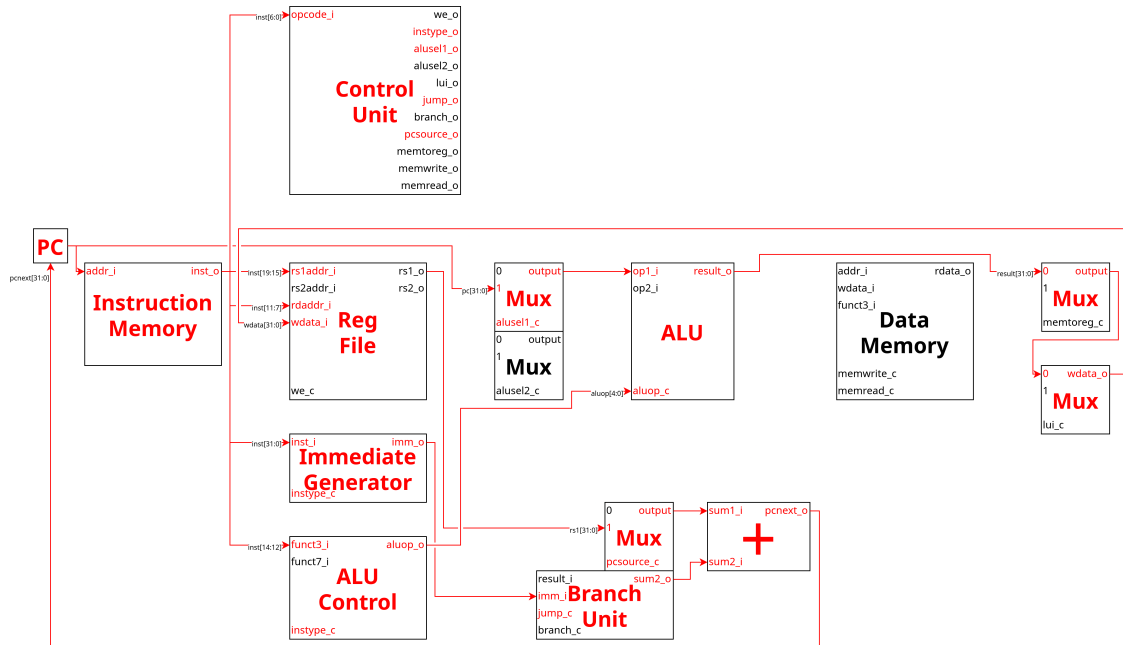


Fonte: O Autor

4.3.8 Caminho de Dados da Instrução de Salto Indireto (*jalr*)

A execução do *jalr* assemelha-se estruturalmente ao *jal*, porém com uma diferença crucial no cálculo do endereço de destino. Em vez de utilizar o PC como base, o endereço do salto é calculado pela ALU somando o valor lido do registrador *rs1* com o imediato de 12 bits com sinal estendido. O endereço de retorno ($PC + 4$) é preservado e gravado no registrador destino (*rd*), e o PC é atualizado com o novo endereço calculado, permitindo saltos dinâmicos baseados em dados armazenados.

Figura 13 – Caminho de Dados da Instrução de Salto Indireto

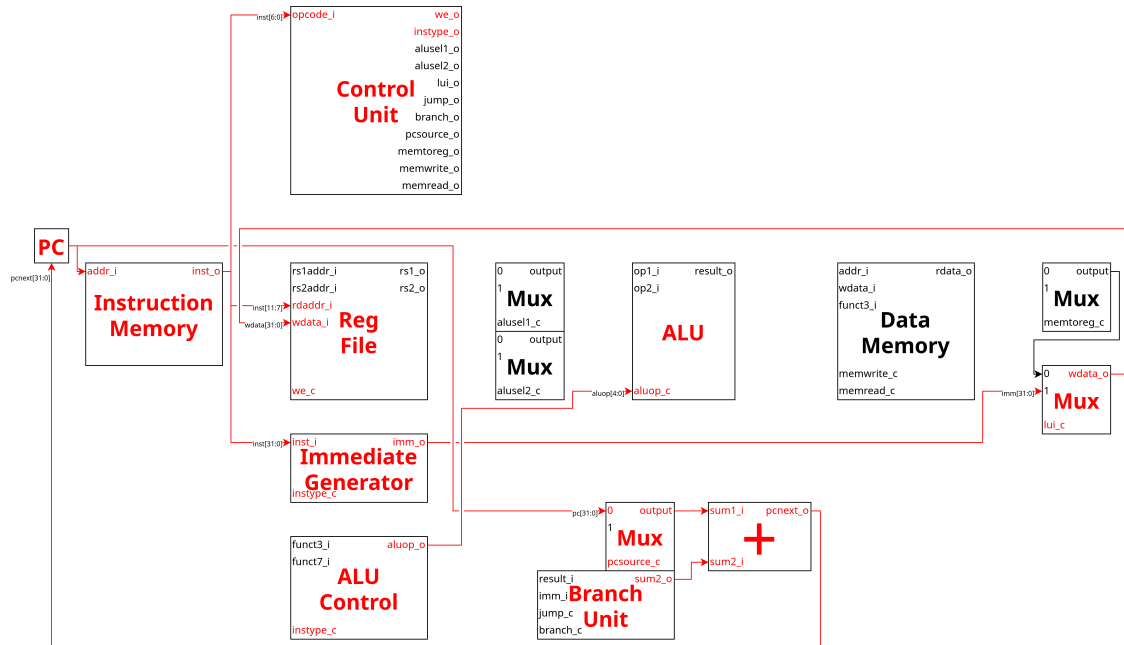


Fonte: O Autor

4.3.9 Caminho de Dados de Carregamento de Imediato Superior (*lui*)

Para a instrução *lui*, o datapath opera de maneira simplificada. O Gerador de Imediatos extrai os 20 bits superiores da instrução, preenche os 12 bits inferiores com zeros e envia este valor de 32 bits pelo caminho de dados e o resultado é imediatamente conduzido ao Banco de Registradores para ser gravado em *rd*. Não há acesso à memória de dados nem leitura de registradores de origem.

Figura 14 – Caminho de Dados de Carregamento de Imediato Superior

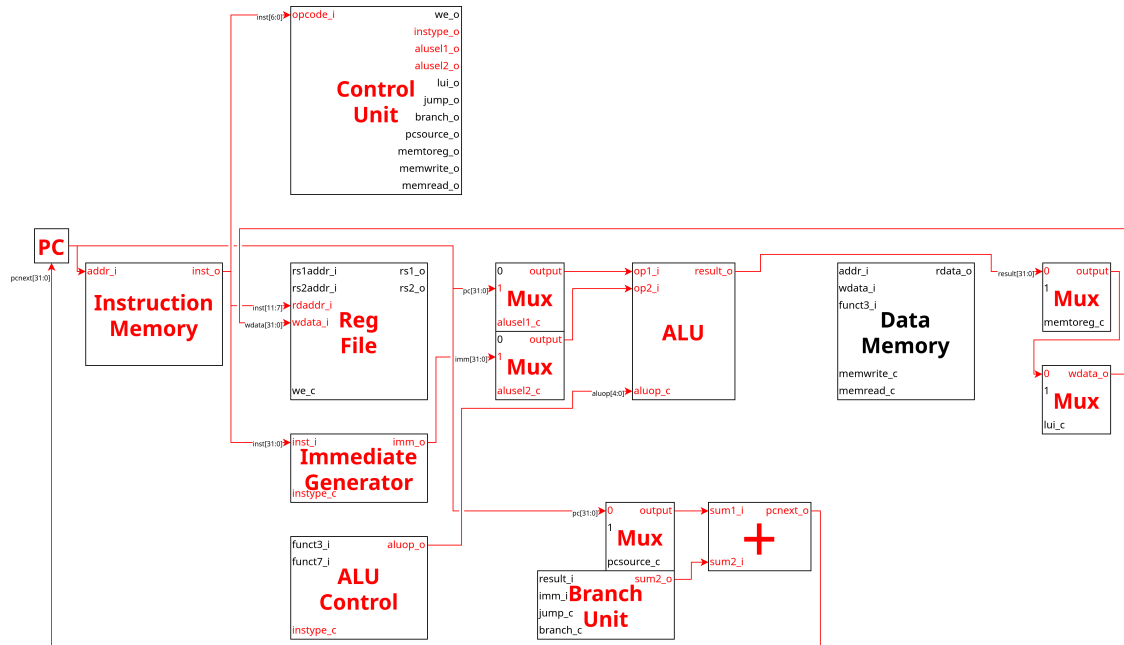


Fonte: O Autor

4.3.10 Caminho de Dados da Adição de Imediato Superior ao PC (*auipc*)

Por fim, o fluxo da instrução *auipc* combina o valor atual do PC com uma constante. O imediato de 20 bits é extraído e posicionado nas casas superiores de uma palavra de 32 bits (com os bits inferiores zerados). A ALU recebe em suas entradas o valor do PC e este imediato formatado, realizando a soma de ambos. O endereço resultante desta adição é então roteado diretamente para a porta de escrita do Banco de Registradores, sendo salvo em *rd* para utilização futura pelo programa.

Figura 15 – Caminho de Dados da Adição de Imediato Superior ao PC



Fonte: O Autor

5 Considerações Finais

Em resumo, o trabalho desenvolvido ao longo deste capítulo cumpriu com êxito os objetivos propostos para esta etapa inicial do projeto. Foi possível estabelecer todas as especificações teóricas e a organização de um processador baseado na arquitetura RISC-V, aplicando na prática os conhecimentos adquiridos na disciplina de Arquitetura e Organização de Computadores. Durante o processo, definiu-se o conjunto de instruções e os seus modos de endereçamento, realizando as adaptações necessárias na arquitetura original para atender às demandas específicas do projeto, como a readequação das instruções de salto. Além disso, a estrutura dos blocos funcionais foi definida com clareza, o que permitiu mapear e especificar o caminho de dados exato percorrido por cada instrução. Com todos esses marcos alcançados, o projeto consolida a base teórica e lógica necessária para avançar com segurança para a sua fase final, que consistirá na implementação do processador utilizando uma linguagem de descrição de hardware.

Referências

PATTERSON, D. A.; HENNESSY, J. L. *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. 1. ed. [S.l.]: Morgan Kaufmann, 2018. Citado na página 7.

RISC-V INTERNATIONAL. *RISC-V Instruction Set Manual, Volume I*. 2024. Acesso em: 04 abril 2026. Disponível em: <riscv.org>. Citado 3 vezes nas páginas 14, 15 e 16.

STALLINGS, W. *Arquitetura e Organização de Computadores*. 10. ed. São Paulo: Pearson, 2017. Citado na página 12.

ZICOG. *Is JAL a J-Type or U-Type instruction? Manual is contradictory on that*. 2018. Acesso em: 04 abril 2026. Disponível em: <<https://github.com/riscv/riscv-isa-manual/issues/152>>. Citado na página 20.